CLASE 5 · 3 HS

Bits y Máscaras

Tratar los 8 LEDs como **un solo byte**. Al salir, el entrenador funciona como un **registro de 8 bits**.

Programación en C con ESP32 · Sistemas Lógicos · Universidad Nacional de Santiago del Estero · 2026

HOY

Seis bloques, un registro de 8 bits

1 · 25 min

Un byte = 8 LEDs

2 · 35 min

Operadores bit a bit

3 · 25 min

Desplazamientos

4 · 35 min

Máscaras

5 · 10 min

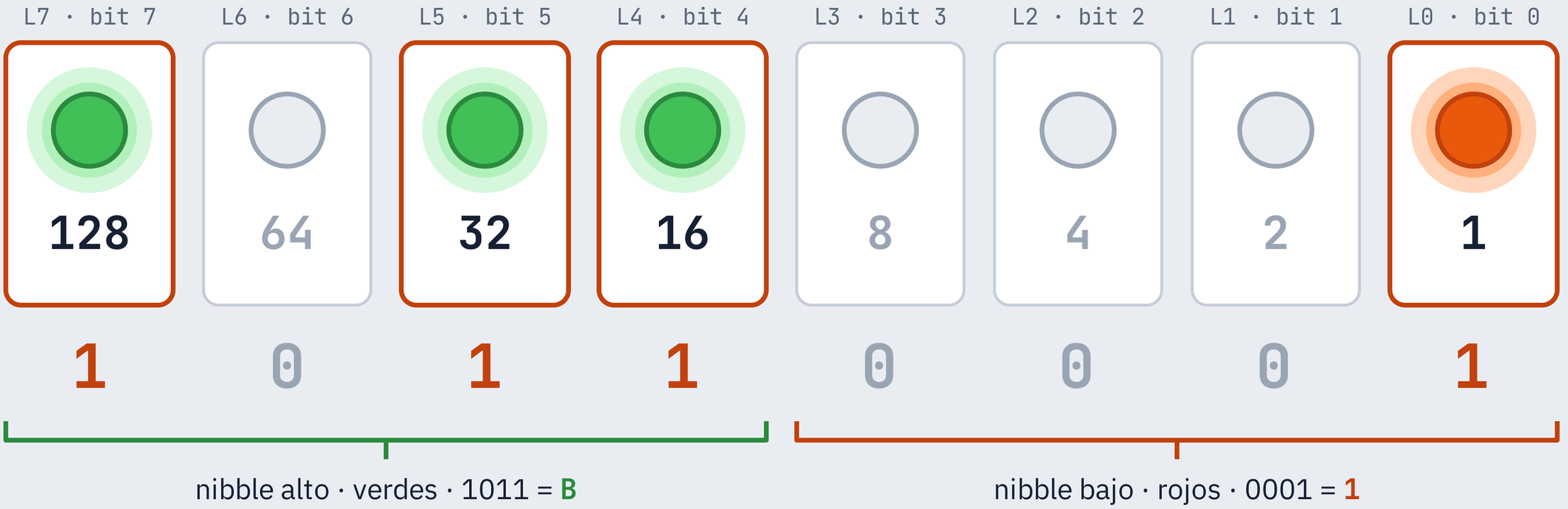
Pulsadores provisorios

6 · 50 min

Ejercicios y TP

1 · UN BYTE = 8 LEDS

Un uint8_t son 8 bits, uno por LED




```
0b10110001 = 0xB1 = 128 + 32 + 16 + 1 = 177
```

El mismo número, tres formas de escribirlo

```
uint8_t a = 177;          // decimal
uint8_t b = 0b10110001;   // binario: 0b
uint8_t c = 0xB1;         // hexa: 0x
```

0b 1011 0001

B 1

Agrupando de a 4 bits: **0xB1** = 128 + 32 + 16 + 1 = **177**

0b Cuando importan los **bits sueltos**: qué LED enciende.

0x Para escribir **un byte entero** de forma compacta.

Dos funciones para todo el curso

```
void mostrarByte(uint8_t valor) {
  for (int i = 0; i < 8; i++) {
    digitalWrite(LEDS[i], (valor >> i) & 1);
  }
}

void imprimirBinario(uint8_t valor) {
  for (int i = 7; i ⩾ 0; i--) {
    Serial.printf("%d", (valor >> i) & 1);
  }
}
```

mostrarByte

Bit i → LED Li. Los 8 bits a los 8 LEDs.

imprimirBinario

printf no tiene formato para binario. Recorre **al revés**, de 7 a 0: en papel el bit de más peso va primero.

(valor >> i) & 1 = "el bit i de valor". En un rato se entiende por qué.



Ocho compuertas a la vez

Operador	Compuerta	Regla por bit
x & y	AND	1 solo si **los dos** son 1
x | y	OR	1 si **alguno** es 1
x ^ y	XOR	1 si son **distintos**
~x	NOT	invierte **cada** bit

```
x       1 1 0 0 1 1 0 0
y       1 0 1 0 1 0 1 0
        ---------------
x & y   1 0 0 0 1 0 0 0
x | y   1 1 1 0 1 1 1 0
x ^ y   0 1 1 0 0 1 1 0
~x      0 0 1 1 0 0 1 1
```

x & y entre dos bytes es un integrado con **8 compuertas AND**, como el 74LS08. Ninguna columna se entera de las otras: no hay acarreo como en la suma.

El error clásico: & no es &&

Lógicos (Clase 3)	Bit a bit (hoy)
&& || !	& | ^ ~
Miran el valor entero: verdadero (≠ 0) o falso (0)	Operan **cada bit** por separado
Resultado: siempre 0 o 1	Resultado: otro byte
Para **condiciones** de un if	Para **manipular bits**

```
5 && 2   // = 1   verdadero
5 & 2    // = 0   101 & 010
!5       // = 0   NO verdadero
~5       // = -6  ¡32 bits!
```

⚠ Regla: **condiciones con &&, bits con &.** Y ~ trabaja en int (32 bits): guardar el resultado en un uint8_t, o %X imprime **FFFFFF33** en lugar de 33.

Correr bits: << multiplica, >> divide

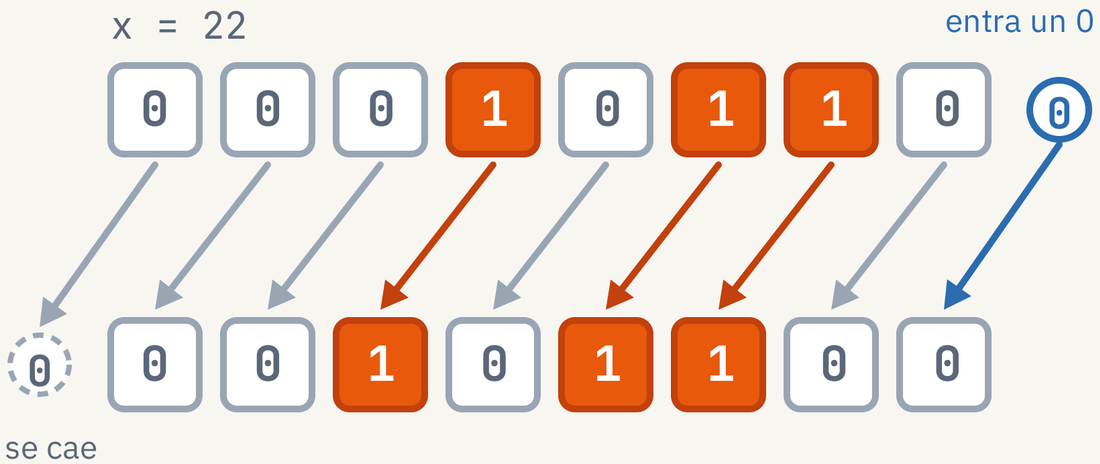
```
x       = 0 0 0 1 0 1 1 0   22
x << 1  = 0 0 1 0 1 1 0 0   44
x >> 1  = 0 0 0 0 1 0 1 1   11
```

Operación	Equivale a	Ejemplo
x << 1	x * 2	22 << 1 = 44
x << n	x * 2^n	3 << 4 = 48
x >> 1	x / 2 (división entera)	22 >> 1 = 11
x >> n	x / 2^n	200 >> 4 = 12
1 << n	2^n: un solo 1, en la posición n	1 << 5 = 0b00100000

En decimal, ×10 es agregar un cero. En binario, **×2 es correr un lugar**. Y x >> 1 es el **registro de desplazamiento** que se verá con flip-flops.

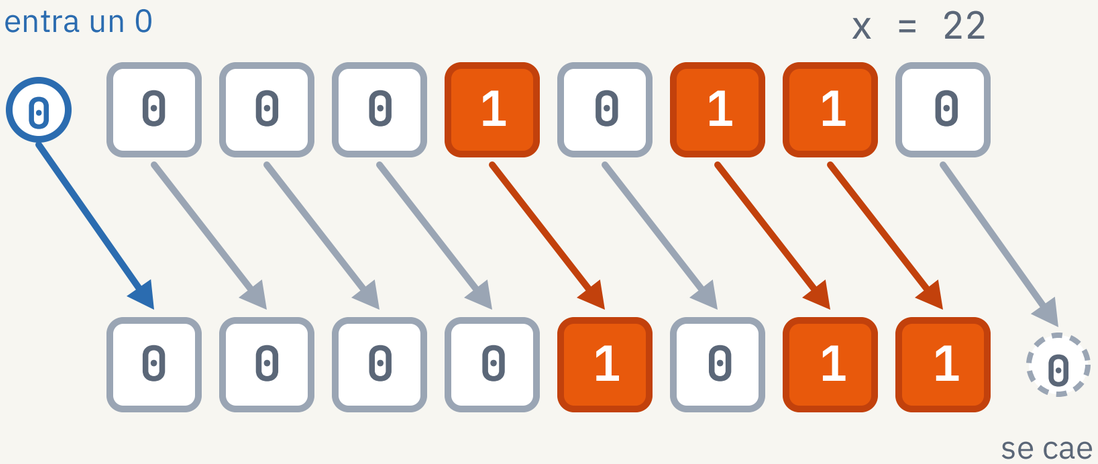
Cada bit pasa a la celda vecina

x << 1 · todo un lugar a la izquierda



x << 1 = 44 · ×2

x >> 1 · todo un lugar a la derecha



x >> 1 = 11 · ÷2

Los bits no cambian: cambian de **lugar**, y cada lugar vale el doble que el de su derecha.

Ahora sí: (valor >> i) & 1

```
valor           = 1 0 1 1 0 0 0 1
valor >> 4      = 0 0 0 0 1 0 1 1
(valor>>4) & 1  = 0 0 0 0 0 0 0 1
```

Correr **i** lugares deja el bit i en la posición 0. El **& 1** borra todos los demás.

```
uint8_t patron = 0b00000001;
for (int i = 0; i < 8; i++) {
  mostrarByte(patron);
  delay(200);
  patron = patron << 1;
}
```

Un 1 que camina por los LEDs: 1, 2, 4, 8… **¡se duplica!** Después de 8 shifts se cae y queda 0.

Rotación circular: lo que sale por un lado entra por el otro

```
x = (x << 1) | (x >> 7);    // rotar a la izquierda (8 bits)
```

```
x           = 1 0 1 1 0 0 0 1
x << 1      = 0 1 1 0 0 0 1 0   el bit 7 se perdió al guardarlo en 8 bits
x >> 7      = 0 0 0 0 0 0 0 1   … pero lo rescatamos aquí
OR de ambos = 0 1 1 0 0 0 1 1   rotado
```

Con solo **x << 1**, el 1 de 0b10000000 "se cae" y queda 0. En una rotación no se pierde nada.



Siempre paréntesis con los bits

En C, **+** y **-** se calculan antes que **<<** y **>>**, y **==** se calcula antes que **&**.

Se escribió	C entiende	Resultado
`1 << n + 1`	`1 << (n + 1)`	¿era eso?
`x << 1 | x >> 7`	`(x << 1) | (x >> 7)`	funciona por casualidad
`if (x & 1 == 0)`	`if (x & (1 == 0))` → `x & 0`	**siempre falso**

⚠ Regla del curso: **toda operación de bits va entre paréntesis.** if ((x & 1) == 0)

Cuatro operaciones para tocar un solo bit

Quiero…	Código	Por qué funciona
Leer el bit n	`(x >> n) & 1`	Lo traigo a la posición 0 y borro el resto
Poner en 1 el bit n	`x |= (1 << n);`	A + 1 = 1 · A + 0 = A
Poner en 0 el bit n	`x &= ~(1 << n);`	A · 0 = 0 · A · 1 = A
Invertir el bit n	`x ^= (1 << n);`	A ⊕ 1 = A' · A ⊕ 0 = A

Las máscaras no son un truco de programadores: son **los teoremas del álgebra de Boole** aplicados a 8 bits a la vez.

La máscara decide columna por columna

OR · encender: x | 0xF0



= 0b11110001 = 0xF1

Máscara en **1**: se enciende. En 0: x pasa igual.

AND · apagar L4: x & ~(1 << 4)



= 0b10100001 = 0xA1

Máscara en **0**: se apaga. En 1: x pasa igual.

Apagar L4 sin tocar los demás

```
x              = 1 0 1 1 0 0 0 1
1 << 4         = 0 0 0 1 0 0 0 0
~(1 << 4)      = 1 1 1 0 1 1 1 1  ← la máscara
x & máscara    = 1 0 1 0 0 0 0 1  ← solo cambió L4
```

~(1 << 4) tiene todo en 1 menos el bit 4. Donde hay 1, el AND deja todo igual; donde hay 0, apaga.

Operadores compuestos

Corto	Largo
`x ⊨ m;`	`x = x | m;`
`x &= m;`	`x = x & m;`
`x ^= m;`	`x = x ^ m;`
`x ⟸ 1;`	`x = x << 1;`
`x ⟹ 1;`	`x = x >> 1;`

Máscaras de varios bits: los nibbles

```
uint8_t bajo = x & 0x0F;          // 0000 1111 → me quedo con L3..L0
uint8_t alto = (x >> 4) & 0x0F;   // traigo L7..L4 abajo y me quedo con ellos
x |= 0xF0;                        // enciendo los 4 verdes sin tocar los rojos
x ^= 0xFF;                        // invierto los 8 (igual que x = ~x)
```

x = 0xB1

1011 0001

alto · L7..L4

0xB = 11

bajo · L3..L0

0x1 = 1

Contar de 0 a 255 es casi nada

```
uint8_t contador = 0;    // global

void loop() {
  mostrarByte(contador);
  imprimirBinario(contador);
  Serial.printf("  0x%02X  %3u\n", contador, contador);
  contador++;             // 255 + 1 = 0 en un uint8_t
  delay(100);
}
```

Trampa

```
for (uint8_t n = 0;
     n ⩽ 255; n++)
```

Nunca termina: un uint8_t siempre es <= 255. Para un for de 0 a 255, usar **int**.

Un truco provisorio para los botones

```
bool a = !digitalRead(BOTON_A);  // apretado = LOW

if (a) {
  registro++;       // la acción
  delay(250);       // para no repetirla mil veces
}
```

Sin el delay

loop() pasa miles de veces mientras el dedo está apretado: la acción se repite miles de veces.

⚠ Es **provisorio**: si se mantiene apretado, se repite cada 250 ms, y durante el delay el micro no hace nada más. En la Clase 7: **detección de flanco**.

6 · EJERCICIOS · 50 MIN

Ahora ustedes

EJERCICIO 1

Contar unos y paridad

contarUnos(x) recorre los 8 bits con (x >> i) & 1. Para datos de 7 bits, poner la paridad par en el bit 7: dato | (paridad << 7).

EJERCICIO 2

Auto fantástico

Un único LED que va de L0 a L7 con <<= y vuelve con >>=, sin parar. Que los extremos no se queden dos turnos.

EJERCICIO 3 · BREVE

Complemento a 2

Para x de 1 a 6: x, ~x y ~x + 1 en binario. Guardar ~x + 1 en un int8_t e imprimirlo con %d. ¿Qué número aparece?

EJERCICIO 4

Nibbles

Separar x = 0xB1 en x & 0x0F y (x >> 4) & 0x0F, en binario, hexa y en los LEDs. ¿En qué se diferencia de x & 0xF0?

TP CLASE 5

Registro de 8 bits

1. Variable global uint8_t registro, siempre visible en los 8 LEDs.
2. Encabezado, LEDS[8] y los tres botones BOTON_A, BOTON_B, BOTON_C.
3. mostrarByte e imprimirBinario, y uint8_t rotarIzquierda(uint8_t x).
4. Arranca en un valor a elección, por ejemplo 0b10110001.
5. Cada cambio se muestra en los LEDs y en binario, hexa y decimal. Patrón provisorio con delay(250).

Botón	Acción	En C
A	Rotar a la izquierda	`rotarIzquierda(…)`
B	Invertir todos los bits	`~registro`
C	Sumar 1	`registro++`

```
Bin: 10110001  Hex: B1  Dec: 177
[A] rotar    → Bin: 01100011  Hex: 63  Dec: 99
[B] invertir → Bin: 10011100  Hex: 9C  Dec: 156
[C] sumar 1  → Bin: 10011101  Hex: 9D  Dec: 157
```

Entrega: carpeta **SL2026-Apellido-Clase5** en ZIP, sin la carpeta .pio. Para pensar: ¿qué queda si se aprieta A ocho veces? ¿Y B dos veces?



CHECKLIST DE SALIDA

Al terminar la clase, sé…

- escribir un mismo byte en decimal, binario (0b) y hexa (0x)
- explicar la diferencia entre & y && con un ejemplo donde den distinto
- calcular a mano &, |, ^, ~, << y >> entre dos bytes
- leer, poner en 1, poner en 0 e invertir un bit con máscaras
- explicar por qué x << 1 multiplica por 2
- usar mostrarByte e imprimirBinario y explicar cada línea

La semana que viene: hoy guardamos un byte. En la Clase 6 guardamos **una lista de bytes**, y con eso armamos un **decodificador de 7 segmentos**.

